

Relatório técnico GeoLog

Plataforma de telemetria logística e persistência poliglota

Instituição UNIPÊ

Disciplina Implementação de Banco de Dados NoSQL

Professor Me. Ricardo Roberto de Lima

Estudantes

José Davi Maurício Oliveira

Vitória Luclécia da Conceição de Oliveira

Everaldo Veloso Calvacanti Júnior

Objetivo e solução

O GeoLog integra informações cadastrais da frota e dados de sensores da LogiTech Express. A solução utiliza React/Vite no frontend, FastAPI no backend, SQLite para motoristas e veículos e MongoDB para telemetria. Essa divisão permite consultar posições por proximidade e reunir cadastros e medições em uma visão unificada. A validação automatizada do backend resultou em 38 testes aprovados, sem falhas ou testes ignorados.

1 Arquitetura poliglota

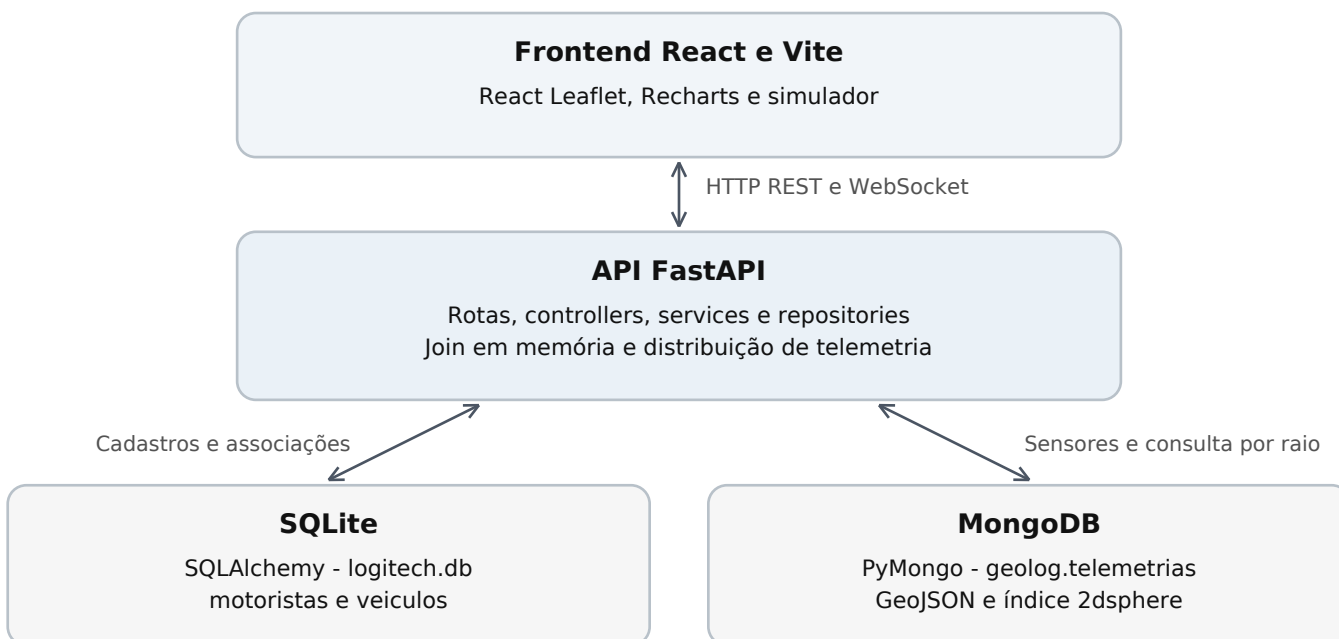


Figura 1. A API centraliza o acesso aos dois bancos. O frontend recebe dados enriquecidos por HTTP e novas telemetrias por WebSocket.

Responsabilidades e modelo de dados

No SQLite, **motoristas** contém id, nome, cnh e status; **veiculos** contém id, placa, modelo e motorista_id. A associação entre as tabelas usa motorista_id. CNH e placa têm restrições de unicidade no modelo SQLAlchemy.

No MongoDB, a coleção **geolog.telemetrias** armazena veiculo_id, location, temperatura, velocidade e timestamp. A localização segue GeoJSON: **type = Point** e **coordinates = [longitude, latitude]**. O documento guarda somente o identificador do veículo; os dados cadastrais são obtidos do SQLite ao montar a resposta.

Funcionamento e implementação

2 Inicialização e consulta geoespacial

Na inicialização, o backend carrega a configuração, abre as conexões e cria as tabelas relacionais. O código de conexão MongoDB cria os índices **location_2dsphere** e **veiculo_timestamp_desc**, este último sobre **veiculo_id** e **timestamp** decrescente. O script **seed.py** fornece três motoristas, três veículos e três telemetrias iniciais.

A rota **GET /api/v1/veiculos/proximos** recebe latitude, longitude, raio em quilômetros e **socket_id**. O pipeline inicia com **\$geoNear** e calcula a distância esférica em metros. Em seguida, ordena os registros por veículo e data, agrupa com **\$group** e seleciona a telemetria mais recente. Só depois aplica o limite **raio x 1000**. Essa ordem evita que uma posição antiga dentro do raio mantenha um veículo que já saiu da região nos resultados.

3 Integração dos dados e dashboard

O repositório de telemetria recupera cada documento MongoDB e consulta o veículo e o motorista correspondentes no SQLite. A API retorna uma estrutura com telemetria, veículo e motorista aninhados. O frontend seleciona a maior data de cada veículo e apresenta nome do motorista, placa, última temperatura, velocidade e coordenadas.

O React Leaflet desenha os veículos e o círculo do raio selecionado; marcadores verdes identificam veículos dentro da área e cinza identifica os demais. O Recharts apresenta o histórico de temperatura por placa e a distribuição do status dos motoristas. Os KPIs consideram a última medição de cada veículo na frota carregada, independentemente do filtro de raio.

Indicador inicial	Valor esperado com o seed
Frotas ativas	2 veículos com motorista no status Ativo
Temperatura média	$(4,2 - 18,5 + 22,0) / 3 \approx 2,6 \text{ }^{\circ}\text{C}$
Alertas de velocidade	1 veículo acima de 80 km/h
Status dos motoristas	2 Ativo e 1 Em Descanso

4 Simulação e atualização em tempo real

O botão **Simular Movimentação** gera pequenas variações de coordenadas, temperatura e velocidade e envia novos registros por **POST /api/v1/telemetrias**. O timestamp é gerado em formato ISO 8601; a velocidade é limitada a valores não negativos. O modo automático repete a simulação a cada três segundos. A API distribui por WebSocket as telemetrias dentro do raio registrado pela sessão; após a simulação, o frontend também recarrega o histórico.

Validação e resultados

5 Estratégia de testes

A suíte usa **pytest 8.3.5** e **TestClient** para executar requisições HTTP e conexões WebSocket sem iniciar um servidor Uvicorn externo. Cada teste de integração recebe um SQLite temporário e um banco MongoDB com nome único `geolog_pytest_<uuid>`. O startup real cria os índices, a fixture insere dados conhecidos e o encerramento remove apenas o banco criado pelo teste. As consultas geoespaciais usam MongoDB real, sem substituir `$geoNear` por uma simulação.

Grupo de testes	Quantidade	Resultado
API e persistência SQLite	17	Aprovados
Consulta geoespacial no MongoDB	8	Aprovados
Telemetria e join poliglota	8	Aprovados
Conexão e mensagens WebSocket	3	Aprovados
Cálculo de distância em quilômetros	2	Aprovados
Total	38	0 falhas e 0 ignorados

A execução em **01/10/2026**, com Python 3.12.14 e MongoDB 7.0.16, aprovou os 38 testes nos dois backends entregues. Foram conferidos os índices, a ordem longitude/latitude, a persistência dos dados, as respostas 404 e 422 e a busca por raios de 0, 1, 5 e 10 km. Também foram aprovados os casos de posição mais recente fora do raio, uma posição por veículo e remoção de sessão ao desconectar. Ao final, não restaram bancos de teste.

Correção identificada pelos testes

Dois testes identificaram que o WebSocket não conseguia enviar telemetrias com timestamp porque `model_dump()` mantinha a data como um objeto `datetime`. A alteração para **`model_dump(mode="json")`** converte a data para texto ISO 8601 na mensagem. A gravação MongoDB mantém o timestamp como data BSON. Após a correção, toda a suíte foi aprovada.

Escopo da validação e reprodução

Os resultados comprovam o comportamento do backend nos cenários testados. A renderização do mapa, os gráficos, os cliques e a execução do frontend no navegador exigem uma verificação visual complementar. O fluxo completo com Docker não foi executado no ambiente de validação; os testes foram executados diretamente pelo Python.

Para reproduzir, instalar `requirements-test.txt` e executar **`python -m pytest -v`** em backend com MongoDB disponível. `TESTES.md` detalha a configuração de `TEST_MONGODB_URI` e a alternativa de execução por Docker.

Conclusão e referências

O GeoLog separa cadastros e telemetria, integra as respostas e consulta posições com índice geoespacial. Os 38 testes aprovados verificam esses comportamentos e a correção de serialização de datas no WebSocket.

Referências: enunciado Desafio Integrador GeoLog do professor Me. Ricardo Roberto de Lima; código-fonte dos projetos GeoLog-backend e GeoLog-frontend; relatório de execução `pytest` e suíte `backend/tests` de 01/10/2026.